

ARC-AGI Project: Final Project

Tung Duong
tduong71@gatech.edu

1 AGENT DESCRIPTION

My ARC-AGI agent solves transformation problems by discovering transformation sequences that transform the input mesh into a target output mesh. It works by combining a transformation library, a beam search algorithm, a heuristic scoring system, and inferring parameters that can be obtained or predicted from the training set, specifically:

1.1 Transformation library:

A comprehensive set of predefined transforms that handle various patterns based on problems from sets B, C and D. I can say that I implemented many transforms for Final but they can be divided into the following categories:

- Basic transformations for the entire grid with numpy like rotate (90, 180, 270), flip and split (horizontal and vertical), mirror, crop, etc.
- Color transformations: swap color, replace color, mapping colors, etc.
- Problem-specific transformations like `cover_border_closed_shape`, `create_spiral`, `connect_two_points_with_shortest_path`, etc.

1.2 Beam search algorithm:

The agent explores transition sequences of maximum length 15, with a beam width of 20. I chose such a beam search configuration based on milestones as well as tests in Final. Specifically, for the maximum sequence length and beam width respectively:

- With milestone B: 3 and 5 (because with this milestone, the transformations are quite simple, most of them only need 1 or 2 transformations to find the result)
- With milestone C: 8 and 15 (more complex transformations, especially color-related issues)
- With milestone D: 8 and 15 (this is the milestone where I have to implement the most specific transformations, so I haven't paid attention to it and left it as milestone C when I think it's good enough to work)

- With Final: 15 and 20 (this is probably the best parameter that I have drawn after many tests, it helps me solve 3 more hidden test problems. I tried to adjust smaller and larger for both metrics but the results were just not good and even made the performance decrease a lot)

Beam search works by keeping the top candidates based on the heuristic score, ensuring efficient exploration of transformation sequences for the parameters we have established above to find transformations that are likely to solve the problem while eliminating transformations that do not lead to a result so as not to reduce the performance of the agent.

1.3 Heuristic Scoring System:

Evaluate transformed grids using the weighted criteria below. These weights I adjusted while solving the problems in the previous milestones so that new and old transformations do not conflict and increase the predictability of transformations that may result:

- Shape Compatibility (40%): Matching grid size.
- Cell Matching (30%): Comparing cell values.
- Color Distribution (15%): Assessing color frequency.
- Structural Pattern (15%): Measuring spatial arrangement via center distance.

1.4 Parameter Inference:

Transforms the inference of parameters from the training set to enhance the adaptiveness. For example, for some problems, the output colors are not in the input, we cannot determine based on the output for the test but we can rely on the output from the training set, as in problem 28e73c20, we need to store the green color to draw the spiral from an input with an empty grid.

1.5 How the agent solves the problems:

The agent uses the training set examples and applies the transformations defined above in turn. For each transformation, it is scored (heuristic), compared to the current grid to see if the transformation should continue. These transformations are generated, tested, and used as the Generate & Test method that I used for Small Projects. I also used Means-Ends Analysis plus a point search method to prune redundant transformation sequences that do not lead to the target grid such as determining grid size, cells of similar and different colors, etc. The Beam

Search algorithm also helps improve performance as I mentioned above. Finally, when the best sequence is found, check if it passes all training runs, if so, apply it to the test and return the maximum 3 predictions. However, this condition is too rigid and does not take advantage of having a maximum of 3 predictions for this project. So for Final, I tweaked it by adding and number of failed transformation sequences to the training set. Then sorting them by the number of failed sequences and picking the best 3 sequences for prediction, this really improved the results, especially for hidden problems.

2 AGENT PERFORMANCE

The performance of the agent across all sets (milestones), based on the Gradescope results, is as follows (note that these are the results for the latest version of my agent):

Table 1 — Performance for solving problems in the milestones and Final.

	Milestone B	Milestone C	Milestone D	Final
Training	16/16 (100%)	16/16 (100%)	16/16 (100%)	48/48 (100%)
Test	13/16 (81.3%)	13/16 (81.3%)	13/16 (81.3%)	39/48 (81.3%)
Running time (seconds)	61	199	87	345
Total	29/32 (91%)	29/32 (91%)	29/32 (91%)	87/96 (91%)

Complexity:

- Time Complexity: $O(b^s)$, where $b=20$ (beam width) and $s=15$ (max sequences).
- Space Complexity: $O(b \cdot s)$ for storing candidate sequences.

From the above results for milestones and Final, it shows that the agent can successfully solve the training (100%), however, with the hidden test, the success rate is only 81.25% (39/48). The 3 problems that the agent has not solved for each milestone are what I can predict when implementing the agent and I will talk about it more clearly in the agent struggle section. Looking at the running time, we can see that milestone D has less than milestone C while milestone D might has more complicated problems. This is because the problems in D are better heuristically, eliminating unnecessary transformations. However, for Final, its running time is nearly 6 minutes, which means that with a large number of

problems, the agent will have difficulty in performance and may not even be able to solve it. I tried running with 400 training problems and here are the results: 50/400 solved with elapsed time: 1230.5608 seconds (20.5 minutes). Although the results are quite good and the running time is acceptable, I am not sure if the number of transforms is larger to solve all 400 problems from training and/or 400 problems from evaluation, it can still maintain good performance.

3 AGENT SUCCESSES

Since the test problems are hidden, I will focus on explaining the training sets and it covers almost all possible examples of the test set. As mentioned above, my agent solves the entire training set (for all three milestones as well as Final), I will present four separate problems to explain how the agent handles it.

3.1 General

The general way to solve the problem has been described above in the Agent Description section above. Here I will go into more detail for each stage, then analyze the following four separate problems (one will be in the bonus section).

For each problem, the agent will apply the transformations for that problem in turn. Each transformation will have its own conditions to limit unnecessary transformations. For example, the two-color swap transformation will not be performed if the grid has less or more than two colors (not counting the background color), or the spiral transformation must have an empty grid at the beginning, or the split transformation with a middle line must have an odd number of rows or columns greater than 3, etc. When performing the transformation, the `_get_possible_params` function will get or predict the possible parameters from the training set, for example, for a problem where the output color is not in the input, we need to analyze and get it from the output of the training data.

After each transformation, the agent will save the names of these transformations in sequences and possible parameters of that transformation. For example, a sequence applying grid rotation 90 degrees, flip horizontal, and swap color will be saved as `[('rotate_90', None), ('flip_horizontal', None), ('swap_color', (np.int64(2), np.int64(5)))]`. Of course, the transformations will be checked for validity and will not repeat each other such as rotating multiple times in a row

but only rotating once or returning to the original position will be eliminated, or flipping the grid horizontally twice in a row, etc. Beam search will also be used to limit the maximum number of transformations in a sequence, with the data in the above section being 3, 8, 8, and 15 for milestones B, C, D, and Final respectively.

Next, the agent will apply these transformation sequences to the inputs and compare them with the output of the training set based on the heuristic score. As mentioned above, it scores based on the grid shape, cell comparison, color distribution, and pattern structure to return the corresponding score for each transformation in the sequence. These transformation sequences will be saved with scores that the agent will use to find the transformations with the highest scores. Here, the agent uses the beam width and the heuristic score for the sequences to eliminate unnecessary candidates and only retain potential candidates. Accordingly, the beam width used is 5, 15, 15, and 20 for milestones B, C, D, and Final, respectively.

Finally, after the agent has all the candidates, it will eliminate the same or similar candidates before applying it to all the tests in the training set. Candidates who have tried to solve the tests in the training set are kept and stored along with the number of tests that they failed to solve. The agent then sorts them and selects the three best candidates (transformation sequences) (with the least number of failures) to predict the input test.

That's the detail of how the agent handles the problems. Now let's go into some concrete examples:

3.2 Problem 6430c8c4:

For this problem, the agent evaluates it with different transforms until the horizontal split transform is found. In this transform, it determines if the number of rows is even, it will skip by returning the grid from the input, which means it will have a heuristic of 0 because the output transform is the same as the input. If the number of rows is odd, it will split the grid into two parts, top (A) (orange on the left) and bottom (B) (gray on the right) without taking the separator (blue). The output is a grid of size equal to either A or B. To fill the output, the agent identifies the colored cells that do not appear in either A or B, stores their positions, and fills them into the output grid. The output color selection is based

on the outputs from the training data, the `_get_possible_params` function will process and store it for specific problems, in this case it will store the output color as green. With the positions from the color cell comparison of both A and B plus the output color from the training set, we will fill the output and return it. When calculating the heuristic score for this output, since it is similar to the entire output, it immediately returns a score of 1000 (the highest) without calculating values such as shape, match cells, etc. Then, the beam search will definitely include this transformation string in the search list and add it to the potential candidates. Here, the candidate solves all the training data so he will be the first candidate (predict 1) for the test.

3.3 Problem **b94a9452**:

Similar to the above, the agent will also test the input transforms one by one to see which one is suitable for the problem. For this problem, transforms such as rotate, flip, split, or other specific problems will not have high scores when the output for this problem has different sizes and color positions from the output. Accordingly, the crop transform will help us get the output with the right grid size but the color is not correct. I will use Training Data 1 to explain specifically. The input is a grid containing a square, here the color is red and the inside is yellow. The crop transform will find the border of the color different from the background color (black) and crop it (like what I did with problem 1cf80156). After cropping, it will have a 5x5 grid but still keep the red color on the outside, yellow on the inside as in the input, it just "crops". Since this transform returns a grid that is the correct size for the output, the heuristic score for this transform is quite high as the score for size comparison is 40%, furthermore, although the color is not correct for the output, the color distribution (15%) is correct, meaning the color for the output edges is one color and the color of the 4 middle cells is one color. From there, with a total score of 55% for this transform, it seems to be the top candidate for beam search and will be kept for further transformations. Now the swap color transform will take care of the rest of the work, which is to swap yellow and red with each other to get the desired output. This swap transform requires that there are only two colors (not counting the background color, which is not needed in this case since the output has no background), then just use them to swap. Then the sequence of the transforms would be `[('crop', None), ('swap_color', (np.int64(2), np.int64(4)))]` (color 2 for red and 4 for yellow). Similarly this transform would result in a heuristic score of 1000 and would be

one of the first candidates for prediction. The sequence of transforms like `[('swap_color', (np.int64(2), np.int64(4))), ('crop', None)]` (swapping color first and cropping later) would also be the second candidate for prediction when the beam search width is also increased accordingly. Transformation sequences like `[('mirror', None), ('swap_color', (np.int64(2), np.int64(4))), ('crop', None)]` or `[('rotate_180', None), ('flip_vertical', None), ('crop', None), ('swap_color', (np.int64(2), np.int64(4)))]`, etc., although not necessary, will also be candidates if there are no better candidates, but will be gradually eliminated. So we can see the tight coupling of transformations, heuristics, beam search in pruning unnecessary transformations and aiming for transformations that can lead to the output result.

3.4 Problem [egb4f6fc](#):

This is one of the problems that I have to implement my own transforms to handle it when the basic transformations of the agent cannot handle it correctly or if they do, it will take a long time (requires expanding beam width, sequence length, and possibly brutal force) and affect other transformations as well as performance or even not solve it at all. I named the transform for this problem `_shapes_swap_color_from_pills`. The input is a grid containing a square and there are n number of pills corresponding to the number of shapes in that square. These pills have only 2 adjacent cells horizontally with 2 colors, especially the colors for the pills are not the same. Since that square even the smallest has at least 4 cells, while the pills are 2 horizontal cells and not adjacent in the same row, I can separate them when iterating over the input. I implemented an algorithm similar to the crop algorithm I discussed above (need to ignore the pills) to crop this grid. From there I have a grid with the same size and color distribution in the output. Next, the agent will compare the colors of the pills with the colors of the shapes in that square, because the colors of the pills are not the same, so for each shape that matches the color of one of the two squares of the pill, we will swap its color (similar to the swap color transform but the color is not from the prediction of the training set but from the pills) with the remaining color in the pill. From there, this transform will return a grid for the desired output. Similar to above, the transform will get a high heuristic score and become a candidate for prediction for this problem. Although it does not use transformations to get the output, however, these particular transformations have an advantage that its sequence is only 1 in length:

[['_shapes_swap_color_from_pills', None]], and once it satisfies the candidate's condition with a high score, the other transforms do not need to be performed anymore, which also improves the performance somewhat.

4 AGENT STRUGGLES

The agent successfully solved all the training problems (48/48). For the test problems, it completed 81.25% (39/48) as mentioned above. For each milestone, it failed to solve 3 hidden tests. As mentioned in previous journals, this is because the agent does not handle color issues well enough, for example, if the background color is not black, then some transforms (especially problem-specific ones) cannot be handled well for hidden tests. However, I did not have time to find those issues and I am quite satisfied with the current results. But I am pretty sure that those are the problems where the color between the background and other colors is difficult to judge when there is no difference in the number or distribution of other colors. For example, problems 3de23699, 195ba7dc, cf98881b, e98196ab, etc. Now let's get into some specific problems:

4.1 Problem **cf98881b**:

This problem can now be solved by the agent by splitting the input into 3 parts A, B, and C (excluding the separator, which is the red line here). The output is a grid with the size of the split parts. Then we will fill the colors in the order of A, B, and C with the condition that the colors filled later will not overlap the colors that have been filled. For example, cell (3,0) has been filled yellow by part A, so it will not be filled blue by part C even though it is in part C. Suppose with the hidden test the background color (black) is replaced with another color, the agent is not designed to determine the remaining colors, so it will not accurately locate the yellow, dark red, and blue cells. To more generally solve problems like this, we could improve the agent by identifying which color appears on all 3 dividers, so the agent can identify the background color and solve for it. But as I said, I'm happy with the current results and don't have time to improve the agent for color-related problems like this.

4.2 Problem **913fb3ed**:

Another problem I took from the training data is 913fb3ed (A). This problem is quite similar to problem ce22a75a (B) that the agent solved based on the `_cell_expand` transform. However, when faced with problem A, the agent cannot

make a correct prediction for the output because it cannot solve the color problem. If the entire output of the squares has a core color, and all the edges are a different color, then other transformations such as `_replace_color` or `_swap_color` can handle the color problem after the cells have been processed by the `_cell_expand` transform like problem B. Therefore, even if we swap the color or change the background color after expanding the cells, it will not be able to give a correct prediction. To overcome this weakness of the transformation to solve these problems, we need to save the colors of the output from the training set in pairs, for example green goes with purple, red goes with blue, etc. and after expanding cells, we will add a transforms whose purpose is to change edge color in pairs. And by combining the chain of these two transforms like `[('_cells_expand', None), ('_pair_colors_edges', color_params)]` with `color_params` are the pairs of colors stored from the training set, then the agent can solve this problem and similar problems.

5 AGENT DEVELOPMENT

My approach improved with each milestone. For milestone A, I solved the problem using a human-like approach of solving each problem individually with specific functions, which can be said to be hard coded, but it was definitely inefficient and was also prohibited in this project for the next milestones. For milestone B, which was the milestone where I spent the most time designing the agent. As mentioned above in the Agent Description section, applying the lessons learned in class at that time as well as in mini projects and homework, I used the Generate & Test method to create transforms, used Means-Ends Analysis and heuristic scoring to prune unnecessary transformations in sequences. In addition, using Beam Search also helped the agent improve its performance significantly. For milestone C, when I used the milestone B agent version for milestone C, I solved some problems, similar to when I used the milestone C version for milestone D. This means that some of my transforms were implemented properly to be able to be used to solve unknown problems for the new milestones. However, the number of solutions for the new milestones was not much (about 1 to 2 problems), which also shows that there were still many bad transforms or the basic transforms were not enough. For each milestone, I tweaked parameters such as the length of sequences, beam width for beam search, improved predictions 2 and 3 after having prediction 1 by

calculating more times the transformations failed, and adjusted the weight for parts of the heuristic. Actually, reading about the exemplary reports for milestones as well as the discussions in Ed (class forum) also gave me many good ideas such as DSL, Discrimination Trees, etc. but there are many reasons why I did not change my approach such as I was not really proficient and understood about those problems and my current solution for the milestones was not too bad. Moreover, although not much, I believe that I have applied some of the lessons learned to this project and the result of 87/96 (91%) problems solved by the agent also shows that my approach and design of the agent is not bad.

6 AGENT-HUMAN COMPARISON

6.1 Similarities

Like humans, agents identify patterns in input-output pairs to choose transformations. In addition, both search in parallel with a process of trial and error, exploring multiple solutions and refining based on feedback (experience). Finally, inferring parameters, problems with similar patterns will be applied similarly to new problems.

6.2 Differences

Humans use intuition to quickly focus on promising solutions, while agents search thoroughly through beam search, trying all the transforms, so they are likely to waste effort on less feasible and time-consuming paths.

Humans can come up with new solutions for unfamiliar models, while agents are limited by a predefined transformation library, struggling with situations that are not pre-programmed (problems that require new transforms). This is a human inference that is much better than this agent, especially with highly complex grids. Furthermore, predictions 2 and 3 are often not very effective because most of the success or failure depends on prediction 1 for the agent, while human predictions are often clearly strategic so it can be said that the efficiency will be much higher for each prediction.

However, with large grids such as 30x30 plus many colors and interspersed or messy, the agent is now more powerful in scanning the data and solving the problem when the human eye may be limited in counting the colors of each cell, so it may be confused and lead to undesirable results.

7 APPENDICES

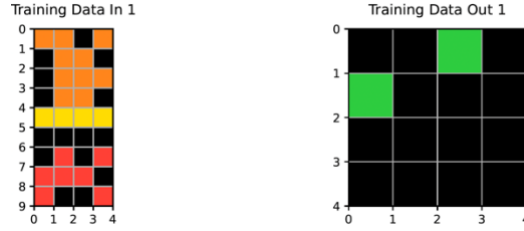


Figure 1— Visual for problem 6430c8c4 (training data 1).

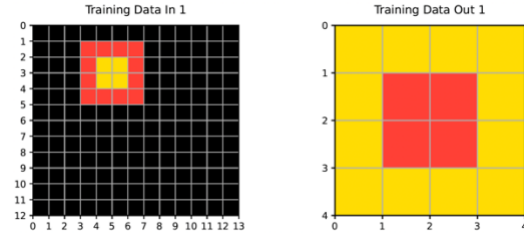


Figure 2— Visual for problem b94a9452 (training data 1).

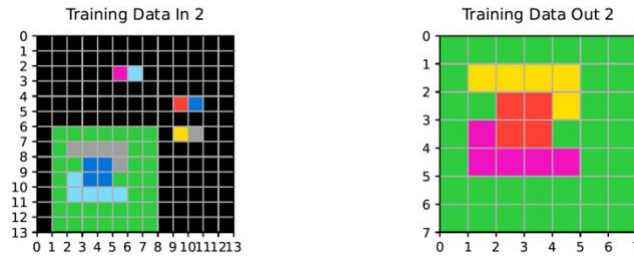


Figure 3— Visual for problem e9b4f6fc (training data 2).

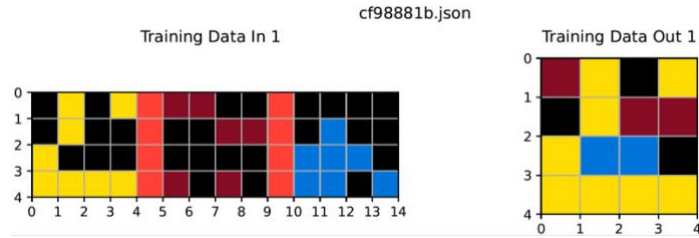


Figure 4— Visual for problem cf98881b (training data 1).

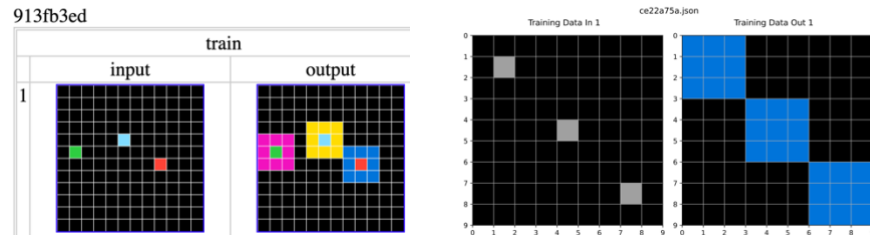


Figure 5— Visual for problem 913fb3ed (A) and ce22a75a (B) (training data 1).

8 BONUS: (MORE PROBLEMS FOR AGENT SUCCESSES)

8.1 Problem 992798f6:

This is also one of the problems that I have to implement my own transforms to handle it. I named this transform `_connect_two_points_with_shortest_path`. The input is a grid of two points (2 cells) with two different colors, here red and blue. Because I need to solve the problem in a more general way when it is necessary to handle similar hidden tests for this problem. So the agent needs to rely on training data to determine or predict. First, determine the number of color cells that appear the most, that is the background color (here is black). Then we will determine the color that connects the two points, that is the color that appears in "out" but not in "in" in the training set. The remaining two colors are two points that are different from the input color (here are red and blue). However, the difficulty is to determine which point is the starting point and which is the ending point. We rely on the output data again, if two cells of the connecting line are in the same row or column, then compare it with two points, the closer point is the end point, the rest is the starting point. Once the color and position of the points have been determined, the agent simply extends the connecting line diagonally towards the end point so that when it cannot (satisfying the condition that it does not coincide with the row or column of the end point), it will continue to extend it horizontally or vertically until it coincides with an adjacent cell of the end point. The reason this problem is difficult to handle with basic transforms is because it cannot know the start and end points, the direction of the connection as well as transformations such as rotation, flip, crop, etc. have no effect on this problem. Similar to the above problem, this transformation also has an advantage in sequence length and a small part in performance.

